Baku Apartment Prices from the Inside

A from-scratch decision tree and Pegasos SVM on 57,421 bina.az listings

Alish Hajiyev · Tahmina Hüseynova · Nihat Ismayilzade · Elshan Samadzade

MLE-AI-201 Machine Learning · AI Academy · October 2026

One real dataset, two learning tasks, two models we had to build ourselves

- **Why it matters:**

Listing prices are heavy-tailed, features mix numbers and Azerbaijani text, and the raw dump is genuinely dirty — the ideal stress test for the classical toolkit.

- **Task A — regression:**

predict log(price). Metrics: RMSE, MAE, R^2 on the log scale and in AZN.

- **Task B — classification:**

premium vs standard, threshold = median price of the training set (208,000 AZN). Metrics: precision, recall, F1, ROC-AUC, confusion matrix.

- **Golden rule:**

both models are our own NumPy code; scikit-learn appears only as the matched baseline.

100,775

raw scraped listings

57,421

clean residential rows

55

engineered features

2 + 2

from-scratch models + bonus baselines

≈ 0.002

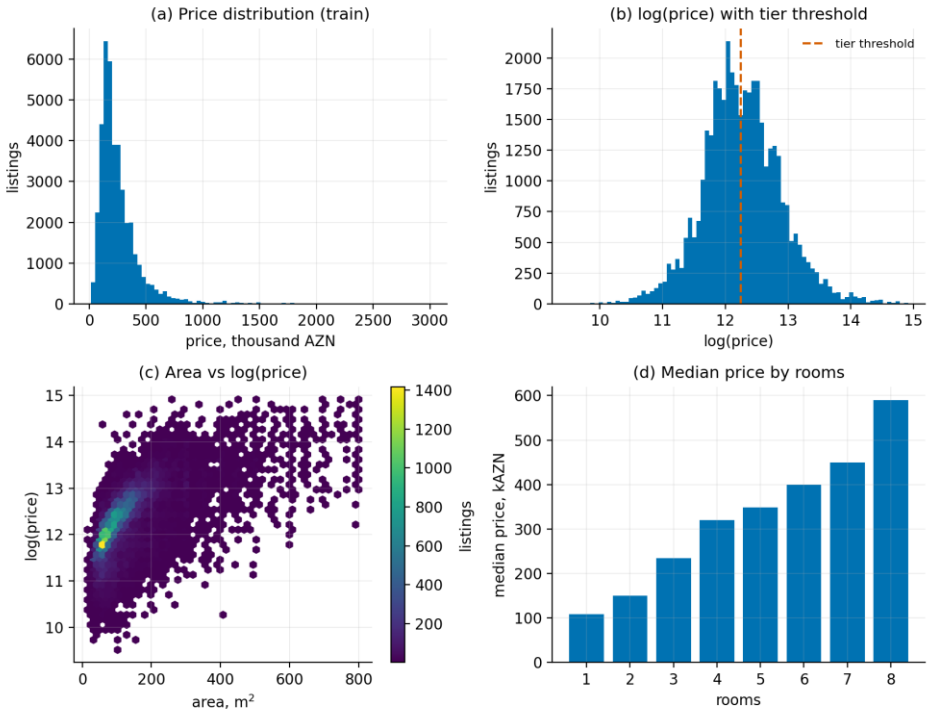
gap to scikit-learn on both tasks (preview of the result)

Takeaway Same features, same split, matched hyperparameters — then the comparison is fair and the number means something.

Cleaning removed 43% of rows — every rule is fixed, documented and reproducible

Step	Rule	Rows
0	Raw Kaggle dump (51 columns, all prices AZN — asserted in code)	100,775
1	Keep the latest scrape per listing URL (scraper revisits)	64,454
2	Residential only: new build, old stock, house / garden house	58,452
3	Parse Azerbaijani strings: “145 m²”, “7 / 9”, “1.3 sot” → numbers	58,452
4	Fixed validity bounds: price 10k–3M, area 15–800 m², coords in AZ	58,110
5	Drop exact repost duplicates	**57,421**

Missing values: constant + explicit “is-missing” indicator. No mean/median imputation, no quantile filters — nothing fitted that could leak from val/test.



Takeaway Rules, not statistics: a fixed bound is a decision made before any split exists, so it cannot leak.

Two columns were arithmetic functions of the target — dropped before any model saw them

Target leakage — dropped

unit_price = price / area
total_price = price restated (equal in 99.99% of rows)

Using them gives a perfect validation score and a useless model.

Also dropped

Identifiers, owner / shop names, address, free text, URLs, scrape timestamps; views / vip / featured — consequences of the listing, not traits of the flat.

55 features kept

Numeric (9): area, rooms, floor, building floors, floor ratio, land area, lat, lng, distance to Fountains Square (haversine).

Binary flags (12): repaired, bill of sale, mortgage, agent-listed, agency, top / ground floor, Baku, has land, and is-missing indicators.

One-hot (34): 3 building categories; 30 most frequent districts + “other”.

Why one-hot, and fitted where?

Both models split on numeric thresholds; ordinal codes would impose a false order on districts.

The only data-dependent encoding choice — which 30 districts get a column — is fitted on the training rows alone.

All categorical text is NFC-normalised first so spelling variants collapse.

Takeaway The statement promised to check for leakage; our feature matrix contains nothing computed from the answer.

Stratified 70/15/15 split; every statistic is learned from the training rows only

TRAIN 70%

40,197 rows

VAL 15%

8,612 rows

TEST 15%

8,612 rows

Stratified on deciles of log(price): each decile is shuffled with seed 42 and cut 70 / 15 / 15.

	train	val	test
median log(price)	12.2453	12.2453	12.2453
premium share	49.9%	49.8%	49.8%

- Tier threshold = train median only (208,000 AZN); val/test reuse it.
- Scaler, district vocabulary, every hyperparameter: fitted / chosen on train + validation.
- Test set touched once per final model. 5-fold CV on the train+val pool refits threshold and scaler inside every fold.
- One command — python -m src.run_all — runs 10 sanity tests, then regenerates every number and figure (seed 42).

Takeaway We cannot stratify on the tier label — it only exists once train and its median exist — so we stratify on price deciles, which matches both tasks.

Our tree scores every candidate threshold of a feature in one vectorised pass

Split objective (sklearn weighting)

$\Delta = N_node/N \cdot [\, I(node) - N_L/N_node \cdot I(L) - N_R/N_node \cdot I(R) \,]$

I = Gini $1 - \Sigma\, p_k^2$ · entropy $-\Sigma\, p_k \log_2 p_k$ · MSE = variance

1 Sort

argsort the feature once per node

2 Accumulate

cumulative class counts (cls) or Σy, Σy^2 (reg) along the sorted order

3 Score

impurity of every left/right prefix in one NumPy expression

4 Pick

largest Δ; threshold = midpoint of the two neighbouring values

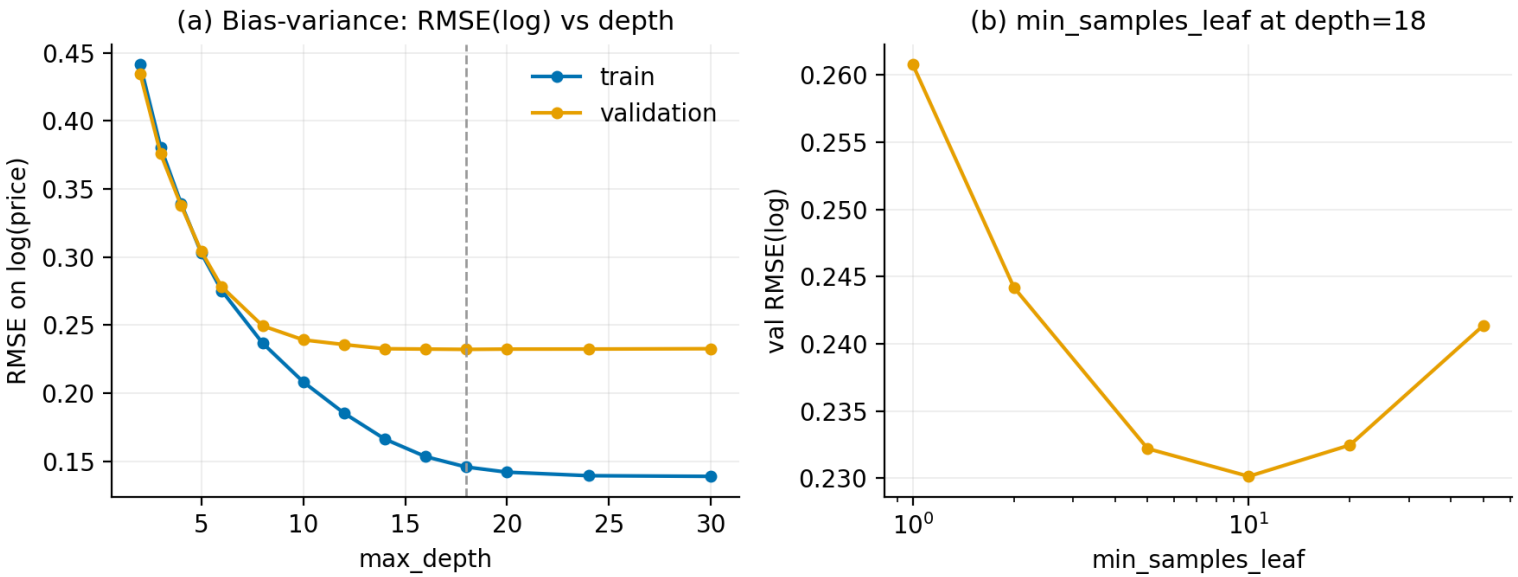
Stopping rules (each unit-tested)

max_depth · min_samples_split · min_samples_leaf · min_impurity_decrease (pruning rule identical to sklearn) · max_features per split (used by the forest bonus)

- Leaves store the mean (regression) or class counts (classification) → predict_proba = leaf frequencies.
- Feature importances = accumulated weighted impurity decrease, normalised.
- Complexity O(F · N log N) per level — the same as scikit-learn; ours is pure NumPy, theirs is Cython (≈12× faster wall-clock).
- Sanity test: on fully-determined toy sets our tree picks the identical feature and threshold as DecisionTreeClassifier.

Takeaway The algorithm is small: ~250 lines of NumPy reproduce the library once the objective and the weighting are implemented exactly.

Validation error bottoms out at depth 18 — deeper trees only memorise the training set



- **Task A (log-price):**

train RMSE keeps falling (0.44 → 0.14) while validation flattens at 18 (0.2322) and drifts up by 30 (0.2327). Chosen on validation: depth 18, min_samples_leaf 10.

- **Task B (tier):**

Gini vs entropy nearly identical (val F1 0.923 vs 0.920); Gini at depth 16 chosen.

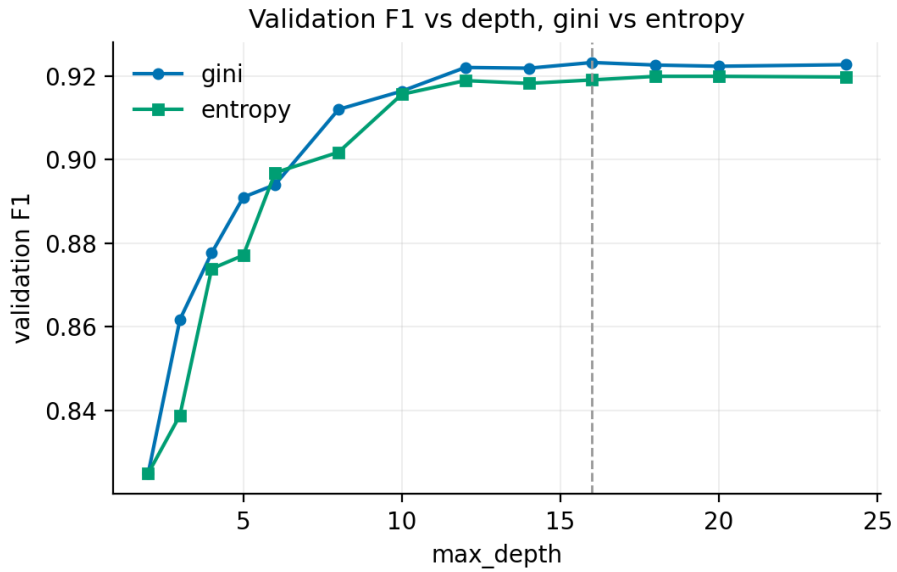
- **min_samples_leaf:**

a clean U-shape — 1 overfits, 50 underfits, 10 is best (panel b).

Takeaway Classic bias–variance: the train–validation gap widens with depth; the chosen configuration sits at the knee, not at the edge of the grid.

With matched hyperparameters our tree reproduces scikit-learn to the third decimal

	Ours	sklearn
Task A — test RMSE (log)	**0.2289**	0.2287
Task A — 5-fold CV RMSE (log)	0.2298 ± 0.0022	0.2295 ± 0.0021
Task B — test F1	**0.9228**	0.9233
Task B — 5-fold CV F1	0.9226 ± 0.0022	0.9224 ± 0.0026
Fit time, 40k rows	≈ 6 s	≈ 0.5 s



- Same objective, same weighting, same stopping rules → same trees up to tie-breaking.
- Residual gap: equal-gain ties (we take the first feature; sklearn permutes features) and last-bit float midpoints.
- CV std ≈ 0.002 ≫ the difference → the agreement is systematic, not one lucky split.

Takeaway The library's edge is engineering (Cython), not statistics — the numbers agree within cross-validation noise.

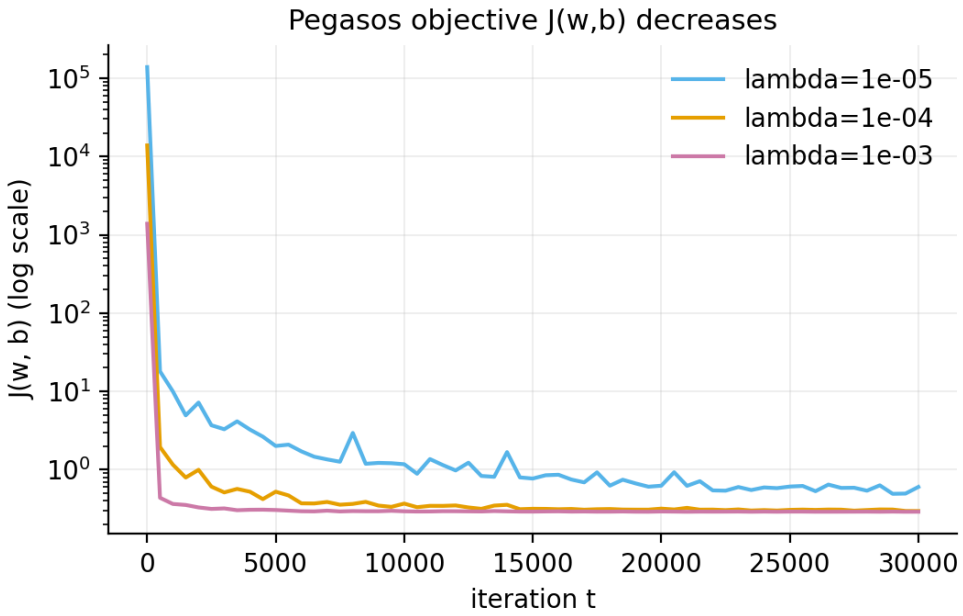
Pegasos: the soft-margin SVM solved by sub-gradient descent — no QP, no SMO

Objective (labels $y \in \{-1, +1\}$)

$$J(w, b) = \lambda/2 \cdot \|w\|^2 + 1/n \cdot \Sigma_i \max(0, 1 - y_i(w \cdot x_i + b))$$

Mini-batch update at step t, $\eta_t = 1/(\lambda t)$, $|B| = 64$

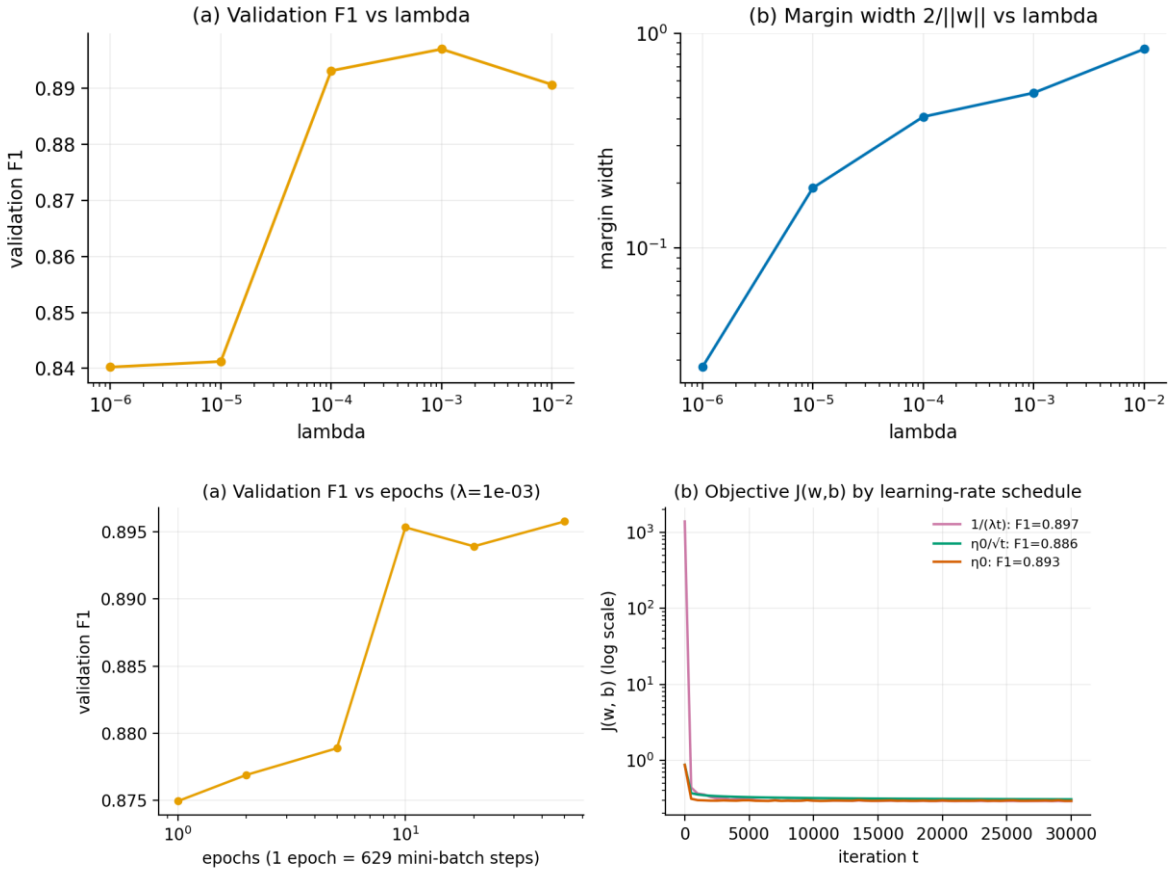
hinge active $(y_i(w \cdot x_i + b) < 1)$: $w \leftarrow (1 - \eta_t \lambda)\, w + \eta_t \cdot \text{mean}(y_i\, x_i)$; $b \leftarrow b + \eta_t \cdot \text{mean}(y_i)$
otherwise: $w \leftarrow (1 - \eta_t \lambda)\, w$
Bias not regularised; features standardised with training statistics (hinge loss is scale-sensitive).



- Support vectors = training points with $y_i(w \cdot x_i + b) \le 1$; margin width = $2/\|w\|$.
- Objective J tracked every 500 steps and shown to decrease (right).
- Non-linearity: random Fourier features $z(x) = \sqrt{(2/D)} \cos(Wx + c)$, $W \sim N(0, 2\gamma I)$, $D = 300$ — an explicit map approximating the RBF kernel, so the same linear solver runs unchanged in O(nD).

Takeaway RFF over kernelised Pegasos: at n ≈ 40k, dual coefficients make every prediction O(n); RFF stays linear-time with one solver.

λ sets the margin–fit trade-off; 1/(λt) converges fastest; F1 plateaus after ~10 epochs



- **λ (regularisation):**

from 10^{-6} to 10^{-2} the margin widens 30× (0.028 → 0.844); validation F1 peaks at $\lambda = 10^{-3}$ — beyond that the model under-fits.

- **Epochs:**

1 epoch = 629 mini-batch steps. F1 is within noise of its plateau after ~10 epochs (0.895 vs 0.896 at 50).

- **Schedule:**

1/(λt) reaches the lowest J (0.291, F1 0.897); constant η_0 is close (0.893); $\eta_0/\sqrt{t}$ shrinks w too slowly — wider margin (0.91) but worse placement (F1 0.886).

- **RFF bandwidth γ:**

best 0.01; larger γ over-fits sharply (F1 0.74 at γ = 0.1).

Takeaway All three solver knobs the statement asks for — λ, epochs, LR schedule — are exposed, and each visibly changes the margin or the metric.

30% of training points are support vectors — the tiers interleave at the threshold; linear beats RBF here

3.79

‖w‖ — norm of the learned weight vector

0.53

margin width 2/‖w‖ (standardised units)

11,917

support vectors of 40,197 (30%) — margin ≤ 1

Task B, test set	F1	ROC-AUC
Ours: Pegasos (linear)	**0.8971**	0.956
sklearn: SGDClassifier(hinge), same λ	0.8989	0.956
sklearn: LinearSVC, C = 1/(nλ)	0.8934	0.955
Ours: Pegasos + RFF (D = 300, γ = 0.01)	0.8491	0.933
sklearn: SVC-RBF (15k subsample)	0.8449	0.926

- Pegasos ≈ its sklearn twin: CV F1 0.8988 ± 0.0016 vs 0.9000 ± 0.0012.
- Both RBF models — ours and sklearn's — lose 5 F1 points to linear: after one-hot encoding 46 of 55 features are binary and the premium boundary is near-linear in area + centrality; the RBF geometry mostly adds variance.
- Two independent RBF implementations agreeing (0.849 vs 0.845) supports that reading.
- SVC-RBF is $O(n^2)$: 16 s to fit on a 15k subsample and 11.5 s to predict — exactly why RFF was the right non-linearity at this scale.

Takeaway A margin you can see: 30% of points inside it tells us the classes genuinely overlap near 208k — the label, not the model, is the limit.

On both tasks the from-scratch models land within 0.002 of their scikit-learn twins

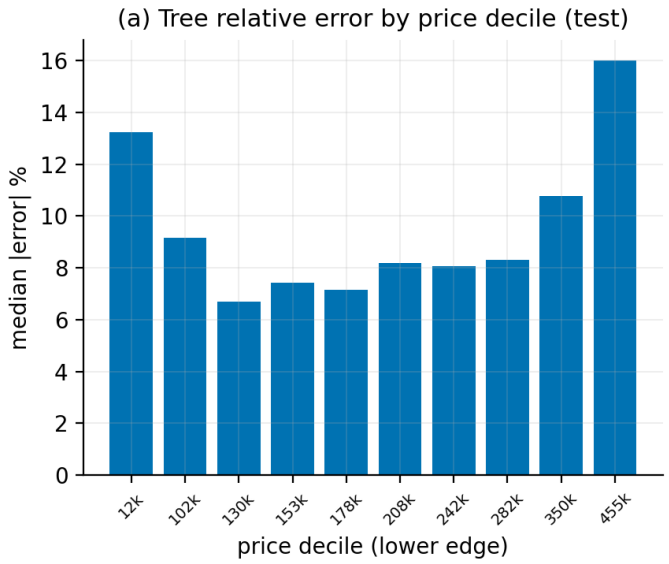
Task A — regression (test)	RMSE log	R^2 log	MAE AZN
Ours: DecisionTree (mse)	**0.2289**	0.865	42,626
sklearn: DecisionTreeRegressor	0.2287	0.866	42,598
Ours: RandomForest ×30 [bonus]	0.2211	0.874	43,132
sklearn: RandomForestRegressor ×30	0.2166	0.879	—
Ours: Ridge, normal equations [bonus]	0.3006	0.768	61,345
Ours: LinearSVR, sub-gradient [bonus]	0.3044	0.762	63,802

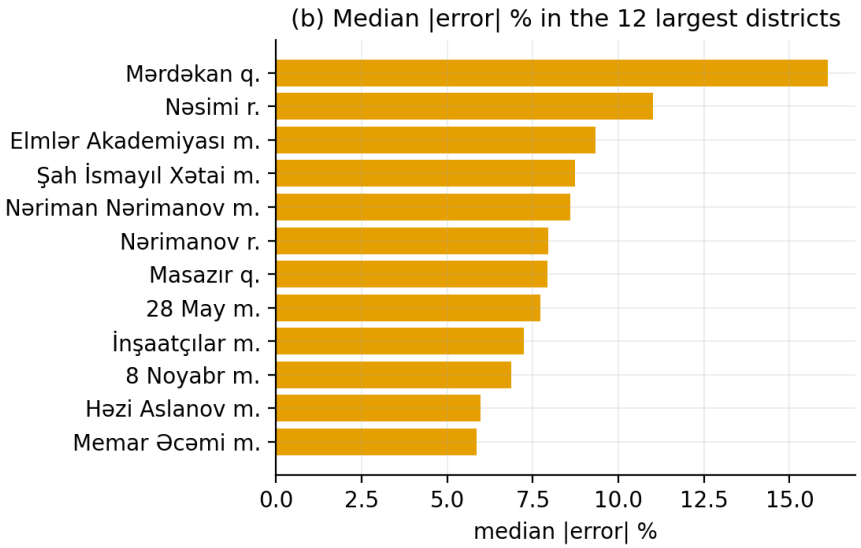
Task B — classification (test)	Acc	F1	AUC
Ours: DecisionTree (gini, 16)	0.923	**0.9228**	0.953
sklearn: DecisionTreeClassifier	0.924	0.9233	0.953
Ours: PegasosSVM (linear)	0.898	**0.8971**	0.956
sklearn: SGDClassifier (hinge)	0.899	0.8989	0.956
sklearn: LinearSVC	0.894	0.8934	0.955
Ours: Pegasos + RFF	0.850	0.8491	0.933
sklearn: SVC-RBF (15k sub)	0.846	0.8449	0.926

Uncertainty (5-fold CV, train+val pool, threshold and scaler refit per fold): tree RMSE 0.2298 ± 0.0022 vs 0.2295 ± 0.0021 · tree F1 0.9226 ± 0.0022 vs 0.9224 ± 0.0026 · Pegasos F1 0.8988 ± 0.0016 vs SGD 0.9000 ± 0.0012. Fit / predict times: our tree ≈ 6 s / 0.04 s, sklearn 0.5 s / 0.003 s; Pegasos 2.2 s; SVC-RBF 16 s / 11.5 s.

Takeaway Regression and classification are reported separately with task-appropriate metrics; no single number declares a winner.

Errors cluster in three places: at the tier threshold, in the price tails, and in the periphery





58% · 62%

tree · SVM misclassifications whose true price is within ±15% of 208k AZN

13% · 16%

median |error| in the cheapest · most expensive decile, vs 7–8% mid-market

- Feature importances and SVM weights agree: area (≈ 59% of impurity decrease), then distance to centre, longitude, district.
- Worst district: Mərdəkan (garden-house suburb) 16% median error; k-means segments: periphery 14–21% vs ≈ 8% in the dense core.
- Both tails are priced on what we do not observe — condition at the bottom; finish, view and prestige at the top.

Classification errors near the threshold are largely irreducible under a median-split framing — the label itself is a coin flip there.

Takeaway The remaining error is explained, not random: label noise at the cut, unobserved attributes in the tails, and sparse peripheral districts.

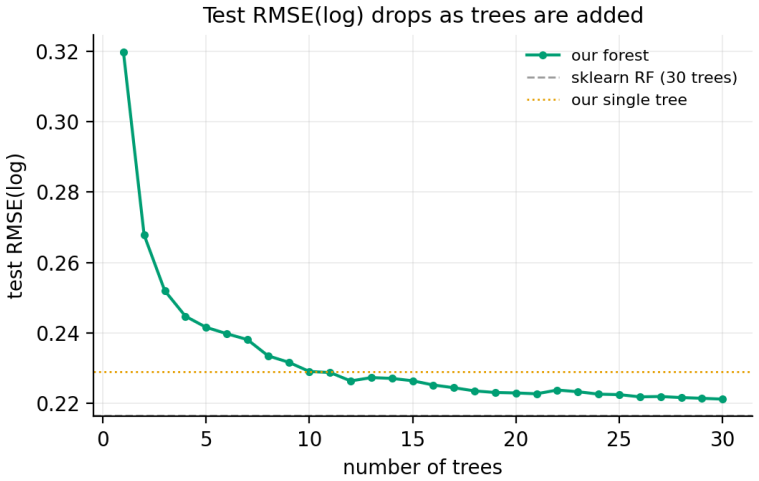
Bonus studies: linear models cannot express area × location; a forest of our trees gains 0.008 RMSE

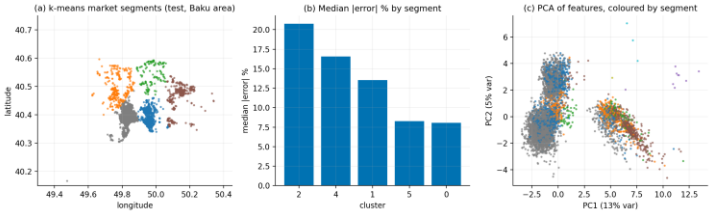
P8 · Ridge + SVR from scratch

Ridge via normal equations (bias unpenalised) equals sklearn to 10^{-15}. Linear SVR by full-batch sub-gradient descent on the ε-insensitive loss equals LinearSVR to 4 decimals (ε chosen on validation).
Both: RMSE_log ≈ 0.30 vs 0.23 for the tree.

P9 · Random forest of our trees

30 bootstrap trees, per-split √d feature subsampling via the tree's own max_features. Test RMSE_log 0.2289 → 0.2211; sklearn's forest 0.2166 with the same settings.





P10 · k-means + PCA market segments

k = 8 on coordinates (fit on non-test rows), PCA of the feature space coloured by segment: core Baku, coastal belt, garden suburbs. Segment error 8% (core) → 21% (periphery) sharpens the error analysis.

Takeaway Each bonus reuses the core code: the forest is our tree with max_features; the SVR is Pegasos with a tube loss; Ridge is the derivation from the exam.

Reproducible by one command, tested against the library — and honest about what the models cannot see

Engineering quality

- Seed 42 everywhere: split, solvers, baselines.
- python -m src.run_all — runs 10 sanity tests, then regenerates every number, table and figure (≈ 10 min).
- Tests pin our tree to sklearn's split on fully-determined examples; Pegasos ≈ LinearSVC; SVR ≈ LinearSVR; Ridge ≡ Ridge; metrics ≡ sklearn to 10^{-9}; split balance.
- Pinned requirements, dataset not committed, cleaning log with row counts, results.json + tables.md as artefacts.
- Modules: data_prep · decision_tree · svm · baselines · ensemble · evaluate · run_all.

Limitations and threats to validity

- Asking prices, not transactions — every model learns seller behaviour.
- 99.7% of rows are Baku; nothing here generalises to other cities.
- Median-split tier makes the label noisy at the cut; ~60% of errors live there.
- exp(log-prediction) estimates the median, biasing AZN metrics for expensive listings (smearing would fix it).
- RFF tuned over a small γ grid with D = 300; SVC-RBF benchmarked on a 15k subsample.
- Free-text descriptions (renovation, view) unused — likely where the luxury-tail signal lives.

Takeaway Every claim on these slides is tied to a file in the repository and a number in results.json.

github.com/thuseynowa/bina-ml-final · tag v1.0-final · Thank you — questions?

The classical algorithms are small — the library's edge is engineering

Tree 0.2289 vs 0.2287 RMSE(log), F1 0.9228 vs 0.9233 · Pegasos F1 0.8971 vs 0.8989 · SVR = LinearSVR

Next: text features · group split by building · quantile forest · smearing-corrected AZN metrics

Alish Hajiyev	Pegasos SVM, LR schedules, margin & support vectors, random Fourier features, SVR bonus
Tahmina Hüseynova	Cleaning rules, leakage analysis, stratified split, run_all orchestrator, EDA & error analysis, report
Nihat Ismayilzade	Metrics from scratch, matched benchmark & 5-fold CV, test suite, Ridge and k-means/PCA bonuses, slides
Elshan Samadzade	Decision tree (vectorised split search, criteria, stopping rules, importances), tree experiments, forest bonus